

Banco Completo de Evaluaciones, Tests y Quizzes — Docker desde Cero (10ma Edición 2026)

Programa de Iniciación Tecnológica (PIT 2026) | OTI - UNI

Docente: Ing. Cristian Jampier Chileno Segundo

Estructura por Sesión:

- **12 Preguntas de Test Asíncrono / Evaluación Formativa** (con opciones múltiples A, B, C, D y solucionario argumentado).
- **6 Preguntas de Quiz de Clase / Pausa Activa** (preguntas rápidas para dinamizar la clase en vivo con solucionario).

Total: 18 Preguntas por Sesión × 6 Sesiones = **108 Preguntas y Respuestas Explicadas.**

SESIÓN 1: Introducción, Virtualización y Primeros Contenedores

Test de la Sesión 1 (12 Preguntas para Aula Virtual)

1. **¿Cuál es la diferencia de arquitectura fundamental entre una Máquina Virtual (VM) y un Contenedor Docker?**
 - a) Las VMs son más ligeras y veloces al arrancar que los contenedores.
 - b) Las VMs emulan un hardware virtual y requieren un Sistema Operativo Invitado completo (Guest OS), mientras que los contenedores ejecutan procesos aislados que comparten el Kernel del Host.
 - c) Los contenedores virtualizan la tarjeta madre y procesadores físicos.
 - d) No existe diferencia de arquitectura; ambas soluciones consumen la misma memoria RAM.
2. **Si ejecutas en la terminal `docker run -d -p 8080:80 nginx`, ¿qué acción realiza exactamente el argumento `-p 8080:80`?**
 - a) Abre el puerto 80 del sistema anfitrión y bloquea el puerto 8080.
 - b) Enruta el tráfico proveniente del puerto 80 del Host hacia el puerto 8080 interno del contenedor.
 - c) Mapea y redirige las conexiones entrantes en el puerto 8080 del Host hacia el puerto 80 interno del contenedor Nginx.
 - d) Asigna 8080 MB de ancho de banda de red al puerto 80.
3. **¿Qué comando de la CLI de Docker se debe utilizar para verificar la lista de contenedores que se están ejecutando en este momento?**
 - a) `docker container status`
 - b) `docker image ls`
 - c) `docker ps` (o `docker container ls`)
 - d) `docker inspect --all`

4. Si deseas detener y posteriormente eliminar un contenedor nombrado `web-demo`, ¿cuál es la secuencia correcta de comandos?
- a) `docker rmi web-demo`
 - b) `docker stop web-demo` y luego `docker rm web-demo`
 - c) `docker kill web-demo` y luego `docker system prune web-demo`
 - d) `docker network rm web-demo`
5. ¿Qué sucede cuando ejecutas `docker run hello-world` por primera vez si la imagen no se encuentra en tu equipo?
- a) El comando genera una excepción y aborta la ejecución.
 - b) El cliente de Docker busca la imagen localmente y, al no encontrarla, la descarga automáticamente de Docker Hub (pull) y la ejecuta.
 - c) Crea una imagen vacía de Python sin dependencias.
 - d) Solicita permisos de administrador para instalar una máquina virtual.
6. ¿Cuál es la función principal de los Control Groups (cgroups) en el motor de Docker?
- a) Asignar direcciones IP fijas a cada contenedor.
 - b) Limitar, auditar y medir el consumo de recursos físicos (RAM, CPU e I/O de disco) de los contenedores.
 - c) Cifrar el sistema de archivos del sistema operativo.
 - d) Crear la interfaz gráfica de usuario en Windows.
7. ¿Qué mecanismo del Kernel de Linux utiliza Docker para aislar la pila de red y las interfaces entre contenedores?
- a) Namespaces PID
 - b) Namespaces MNT
 - c) Namespaces NET
 - d) Namespaces IPC
8. ¿Cuál es el comando correcto para eliminar una imagen local etiquetada como `python:3.12-slim`?
- a) `docker container rm python:3.12-slim`
 - b) `docker image rm python:3.12-slim` (o `docker rmi python:3.12-slim`)
 - c) `docker system clean python:3.12-slim`
 - d) `docker network disconnect python:3.12-slim`
9. ¿Qué beneficio aporta ejecutar un contenedor con el flag `-d` (detached)?
- a) Desconecta el contenedor de la red de internet.
 - b) Ejecuta el contenedor en segundo plano, liberando de inmediato la terminal de comandos.
 - c) Desactiva la lectura de archivos de configuración.
 - d) Inicia el contenedor en modo interactivo con una consola Bash.
10. ¿Bajo qué modelo de arquitectura opera la plataforma Docker?
- a) Monolítica local sin sockets.

- b) Cliente-Servidor (CLI conectada mediante Socket UNIX o API REST con el Docker Daemon).
- c) Red P2P sin servidores centrales.
- d) Servidor SSH sin autenticación.

11. En un **Dockerfile**, ¿qué instrucción define el proceso o comando por defecto que arrancará el contenedor?

- a) **RUN**
- b) **COPY**
- c) **CMD**
- d) **FROM**

12. ¿Por qué motivo Docker Desktop en Windows requiere obligatoriamente el entorno WSL 2?

- a) Para renderizar la interfaz gráfica de las aplicaciones web.
- b) Porque WSL 2 provee un Kernel de Linux real sobre el cual operan los Namespaces y cgroups que necesita Docker.
- c) Para permitir la instalación de paquetes **.deb**.
- d) Para actuar como antivirus del sistema.

Solucionario Explicado — Test Sesión 1

1-b: Las VMs virtualizan hardware y ejecutan un Guest OS completo; los contenedores comparten el Kernel del Host.

2-c: La sintaxis **-p HOST:CONTENEDOR** mapea el puerto 8080 del host al 80 del contenedor.

3-c: **docker ps** o **docker container ls** muestran los contenedores activos.

4-b: Primero se debe detener el proceso con **stop** antes de poder remover el contenedor con **rm**.

5-b: Docker realiza un **pull** automático desde Docker Hub si la imagen no existe localmente.

6-b: **cgroups** es el componente del kernel que limita el uso de RAM, CPU e I/O.

7-c: **NET Namespace** provee aislamiento para interfaces de red, tablas de ruteo e IPs.

8-b: **docker rmi** o **docker image rm** elimina imágenes locales del disco.

9-b: El flag **-d** (detached) corre el proceso en background liberando el prompt de la consola.

10-b: Docker usa arquitectura Cliente-Servidor; el cliente CLI le habla al Docker Engine (Daemon).

11-c: **CMD** establece el comando de arranque por defecto en tiempo de ejecución.

12-b: WSL 2 suministra el Kernel de Linux nativo indispensable para los contenedores en Windows.

Quiz en Vivo de la Sesión 1 (6 Preguntas Rápidas para la Clase)

1. **(Quiz 1.1)** Un alumno dice: "Docker no funciona en mi laptop porque no tengo 16GB de RAM para crear máquinas virtuales". ¿Es correcta su afirmación?

- a) Sí, los contenedores consumen más RAM que una VM.
- b) No, porque los contenedores no emulan un SO completo ni reservan RAM fija; consumen solo los megabytes que el proceso necesita.
- c) Sí, Docker requiere mínimo 32GB de RAM.
- d) No, porque Docker solo corre en la nube.

2. **(Quiz 1.2)** ¿Cuál es la analogía clásica para diferenciar Imagen de Contenedor?

- a) La imagen es el motor y el contenedor es la gasolina.
 - b) La imagen es la receta de cocina (estática) y el contenedor es el plato servido en ejecución.
 - c) La imagen es la pantalla y el contenedor el teclado.
 - d) Son exactamente lo mismo con diferente nombre.
3. **(Quiz 1.3)** Si ejecutas `docker run -it ubuntu bash`, ¿para qué sirven los flags `-it`?
- a) Para instalar paquetes de internet.
 - b) Para abrir una terminal interactiva (i) asignando un pseudo-TTY (t) al contenedor.
 - c) Para reiniciar el contenedor si se cae.
 - d) Para ocultar la consola de comandos.
4. **(Quiz 1.4)** Si tienes un servidor web escuchando en el puerto 5000 dentro del contenedor y usas `-p 8080:5000`, ¿a qué dirección debes ingresar desde tu navegador?
- a) `http://localhost:5000`
 - b) `http://localhost:8080`
 - c) `http://localhost:80`
 - d) `http://127.0.0.1:3000`
5. **(Quiz 1.5)** ¿Qué comando te muestra TODOS los contenedores (tanto los que están activos como los que ya se detuvieron)?
- a) `docker ps`
 - b) `docker ps -a`
 - c) `docker images`
 - d) `docker logs`
6. **(Quiz 1.6)** ¿Por qué se dice que Docker soluciona la frase "En mi PC sí funciona"?
- a) Porque reescribe el código fuente en lenguaje C.
 - b) Porque empaqueta el código junto con sus dependencias y runtime exacto en una unidad inmutable.
 - c) Porque elimina los errores de sintaxis del programador.
 - d) Porque convierte Windows en Linux.

Solucionario Explicado — Quiz Sesión 1

- 1.1-b: Los contenedores comparten el kernel y consumen solo la RAM que requiere el proceso.
- 1.2-b: Imagen = Receta inmutable de solo lectura; Contenedor = Instancia viva en ejecución.
- 1.3-b: `-i` (interactive) y `-t` (tty) permiten interactuar en tiempo real con la consola del contenedor.
- 1.4-b: Se accede por el puerto del Host (8080), el cual redirige internamente al 5000 del contenedor.
- 1.5-b: `docker ps -a` muestra el historial completo de contenedores activos e inactivos.
- 1.6-b: Empaquetar app + runtime + dependencias garantiza reproducibilidad total en cualquier equipo.

SESIÓN 2: Dockerfile Profesional, Imágenes y Docker Hub



Test de la Sesión 2 (12 Preguntas para Aula Virtual)

1. ¿Cuál es la función principal de la instrucción **WORKDIR /app** en un Dockerfile?

- a) Descargar el compilador de Python en **/app**.
- b) Establecer el directorio de trabajo interno donde se ejecutarán las instrucciones posteriores (**COPY**, **RUN**, **CMD**).
- c) Asignar permisos de administrador al directorio del Host.
- d) Limpiar la memoria caché del disco duro.

2. ¿Cuál es la diferencia técnica entre las instrucciones **RUN** y **CMD** en un Dockerfile?

- a) **RUN** se ejecuta en tiempo de compilación de la imagen (**docker build**); **CMD** se ejecuta al iniciar el contenedor (**docker run**).
- b) **RUN** sirve para descargar imágenes de Docker Hub y **CMD** para borrarlas.
- c) **RUN** abre puertos de red y **CMD** configura variables de entorno.
- d) Son instrucciones idénticas y se pueden intercambiar libremente.

3. ¿Por qué la variante de imagen base **python:3.12-slim** es preferible frente a **python:latest** para entornos de producción?

- a) Porque **latest** elimina todas las librerías de seguridad.
- b) Porque **slim** fija una versión concreta reducida que garantiza compilaciones reproducibles, evitando cambios inesperados que genera **latest**.
- c) Porque **slim** solo ocupa 1 MB de disco.
- d) Porque **latest** no permite instalar paquetes con **pip**.

4. ¿Qué utilidad ofrece el archivo **.dockerignore** durante el proceso de **docker build**?

- a) Cifra las contraseñas guardadas en el contenedor.
- b) Evita enviar archivos innecesarios o sensibles (**.git**, **node_modules**, **.env**) al contexto de construcción del motor Docker.
- c) Elimina las imágenes antiguas de la computadora.
- d) Evita que el contenedor tenga acceso a la red de internet.

5. ¿Cómo aprovecha el motor de Docker el sistema de capas y la caché durante la construcción de una imagen?

- a) Borra todas las capas previas y las descarga de nuevo en cada build.
- b) Reutiliza las capas inmutables que no han cambiado, invalidando el caché únicamente a partir de la primera instrucción que sufrió modificaciones.
- c) Cifra las capas en la memoria RAM para que no se puedan modificar.
- d) Ejecuta el compilador en la nube de Docker Hub.

6. ¿Por qué es considerado una MALA PRÁCTICA colocar **COPY . . ANTES de RUN pip install -r requirements.txt**?

- a) Porque marca un error de sintaxis en el Dockerfile.
- b) Porque cualquier cambio en cualquier archivo del código fuente invalidará la caché, obligando a reinstalar todas las librerías en cada build.

- c) Porque **COPY** borra el archivo **requirements.txt**.
- d) Porque **pip** no funciona si los archivos ya están copiados.

7. ¿Cuál es la diferencia práctica entre las instrucciones **COPY** y **ADD**?

- a) **COPY** es la instrucción recomendada para copiar archivos locales; **ADD** además puede descomprimir tarballs **.tar.gz** y descargar URLs.
- b) **ADD** copia archivos locales y **COPY** descarga de internet.
- c) **COPY** solo funciona con archivos de texto y **ADD** con imágenes **.png**.
- d) Ambas son obsoletas y fueron reemplazadas por **MOVE**.

8. ¿Qué diferencia existe entre la instrucción **EXPOSE 5000** y la opción **-p 5000:5000** al ejecutar **docker run**?

- a) **EXPOSE** publica el puerto en internet y **-p** lo documenta.
- b) **EXPOSE** es solo metadato documental en la imagen; **-p** es el comando que realmente abre y mapea el puerto en la máquina anfitriona.
- c) Ambas realizan la misma acción de apertura de red.
- d) **EXPOSE** solo funciona con protocolos UDP.

9. ¿Qué beneficio aporta el patrón de diseño **Multi-stage build** en un **Dockerfile**?

- a) Permite ejecutar múltiples contenedores en paralelo en la misma terminal.
- b) Permite compilar la app en una etapa inicial pesada y copiar únicamente el resultado final a una imagen limpia de runtime, reduciendo el tamaño y vulnerabilidades.
- c) Duplica la memoria RAM del contenedor.
- d) Permite crear imágenes para Windows y Linux al mismo tiempo.

10. Si deseas etiquetar tu imagen local **mi-flask:v2** para subirla a tu cuenta de Docker Hub con usuario **cristian**, ¿qué comando ejecutas?

- a) `docker push mi-flask:v2 cristian`
- b) `docker tag mi-flask:v2 cristian/mi-flask:v2`
- c) `docker rename mi-flask:v2 cristian/mi-flask:v2`
- d) `docker upload mi-flask:v2 --user cristian`

11. ¿Qué comando se debe ejecutar antes de realizar un **docker push** para autenticar tu terminal con tu cuenta de Docker Hub?

- a) `docker connect`
- b) `docker login`
- c) `docker auth`
- d) `docker signin`

12. ¿Por qué la instrucción en forma Exec CMD `["python", "app.py"]` es preferible frente a la forma Shell CMD `python app.py`?

- a) Porque la forma Exec ejecuta el proceso directamente como PID 1, permitiendo recibir señales de apagado limpias del sistema (**SIGTERM**).
- b) Porque la forma Shell no soporta Python 3.
- c) Porque la forma Exec ocupa menos espacio en la imagen.

- d) No hay diferencia de ejecución.

🔑 Solucionario Explicado — Test Sesión 2

- 1-b: `WORKDIR` fija el directorio de trabajo donde operan las instrucciones posteriores.
- 2-a: `RUN` corre comandos en el build de la imagen; `CMD` corre al iniciar el contenedor.
- 3-b: Fijar versiones como `python:3.12-slim` garantiza imágenes ligeras y builds reproducibles.
- 4-b: `.dockerignore` excluye archivos no deseados enviando menos datos al Build Context.
- 5-b: Docker reutiliza capas de la caché hasta encontrar una instrucción o archivo modificado.
- 6-b: Copiar `requirements.txt` e instalar dependencias ANTES de copiar todo el código optimiza el caché.
- 7-a: `COPY` es la opción estándar y segura; `ADD` descomprime tarballs automáticamente.
- 8-b: `EXPOSE` es documental; `-p` efectúa el ruteo de puertos en la red del Host.
- 9-b: Multi-stage separa el entorno de build (pesado) del entorno de ejecución (ligero y seguro).
- 10-b: `docker tag` asigna la nomenclatura requerida por Docker Hub: `usuario/imagen:tag`.
- 11-b: `docker login` autentica la CLI contra el registro remoto de Docker Hub.
- 12-a: La forma `Exec ["binario", "arg"]` ejecuta el proceso como PID 1 sin invocar un subshell intermediario.

⚡ Quiz en Vivo de la Sesión 2 (6 Preguntas Rápidas para la Clase)

1. **(Quiz 2.1)** Un desarrollador guardó su archivo de contraseñas `.env` en la carpeta del proyecto y compiló la imagen sin usar `.dockerignore`. ¿Qué riesgo ocurrió?
 - a) La imagen no se podrá compilar.
 - b) El archivo `.env` fue copiado dentro de la imagen y cualquiera que descargue la imagen podrá ver sus claves con `docker inspect`.
 - c) Docker cifró el archivo `.env` automáticamente.
 - d) La base de datos se borró de la máquina.
2. **(Quiz 2.2)** ¿Qué significa el punto final `.` en el comando `docker build -t mi-app:v1 .`?
 - a) Que la imagen será de tipo privada.
 - b) Que el contexto de construcción (build context) es el directorio actual donde está el Dockerfile.
 - c) Que se descargará de internet.
 - d) Es un error de tipeo y debe eliminarse.
3. **(Quiz 2.3)** Si editas una sola línea de código en `app.py` y vuelves a ejecutar `docker build`, ¿Docker volverá a descargar la imagen base de Python?
 - a) Sí, siempre descarga todo de nuevo.
 - b) No, reutilizará las capas de la imagen base y de las dependencias desde la caché local.
 - c) Sí, a menos que apagues la computadora.
 - d) Solo si usas Windows.
4. **(Quiz 2.4)** ¿Cuál es la diferencia entre las imágenes base `python:3.12-slim` y `python:3.12-alpine`?
 - a) `slim` usa Debian y es muy compatible; `alpine` es ultra liviana (5MB) pero usa `musl C` lo que puede dar problemas de compilación con algunas librerías Python.

- b) **alpine** es solo para computadoras Apple.
- c) **slim** es una versión de prueba y **alpine** de pago.
- d) Tienen exactamente la misma tecnología por dentro.

5. **(Quiz 2.5)** Si ejecutas **docker history mi-imagen:v1**, ¿qué información obtienes?

- a) La lista de usuarios que han descargado la imagen.
- b) El historial de capas que componen la imagen y el tamaño de cada una.
- c) Las páginas web visitadas desde el contenedor.
- d) El código fuente completo descomprimido.

6. **(Quiz 2.6)** ¿Por qué se recomienda no usar la etiqueta **:latest** en los despliegues de producción?

- a) Porque **:latest** cobra una tarifa por descarga.
- b) Porque **:latest** es un puntero dinámico que cambia con el tiempo, lo que puede introducir cambios no probados en producción.
- c) Porque provoca lentitud en el procesador.
- d) Porque Docker eliminará esa etiqueta en el futuro.

Solucionario Explicado — Quiz Sesión 2

2.1-b: Sin **.dockerignore**, archivos sensibles copiados a la imagen quedan expuestos a inspección.

2.2-b: El punto **.** especifica la ruta del Build Context donde el daemon busca los archivos del proyecto.

2.3-b: Las capas anteriores no modificadas se toman instantáneamente desde la caché.

2.4-a: **slim** (Debian) garantiza compatibilidad C estándar; **alpine** (musl C) requiere validar paquetes C.

2.5-b: **docker history** inspecciona cada capa, instrucción y peso de la imagen.

2.6-b: Versionar explícitamente (**v1.0.1**) evita que updates automáticos de **:latest** rompan la app.

SESIÓN 3: Docker Compose y Aplicaciones Multi-Contenedor

Test de la Sesión 3 (12 Preguntas para Aula Virtual)

1. **¿Qué es Docker Compose en el ecosistema de contenedores?**

- a) Un comando nativo del Kernel para crear tarjetas de red físicas.
- b) Una herramienta declarativa para definir, configurar y ejecutar aplicaciones multi-contenedor mediante archivos YAML.
- c) Un compilador de código fuente en lenguaje C++.
- d) Un registro en la nube sustituto de Docker Hub.

2. **¿Cuál es la regla de sintaxis más importante que se debe respetar en los archivos YAML de Docker Compose?**

- a) Colocar punto y coma **;** al final de cada instrucción.
- b) Encerrar todos los valores entre corchetes **{}**.
- c) Respetar estrictamente la indentación usando espacios en blanco (nunca usar la tecla Tabulador).

- d) Escribir todo el documento en letras mayúsculas.
3. En un archivo **docker-compose.yml**, ¿cuál es la sección de nivel superior donde se declaran los contenedores que van a ejecutarse?
- a) **volumes:**
 - b) **services:**
 - c) **networks:**
 - d) **apps:**
4. Si dentro de un Compose tienes un servicio llamado **web** (Flask) y otro llamado **db** (PostgreSQL), ¿cómo debe conectarse Flask a la base de datos?
- a) Usando la dirección IP dinámica de la laptop anfitriona.
 - b) Usando el hostname del nombre del servicio **db**, resuelto por el DNS interno de la red de Docker.
 - c) Usando la dirección de loopback **localhost**.
 - d) Abriendo un canal SSH interactivo.
5. ¿Qué comando de Docker Compose se utiliza para construir las imágenes y levantar todos los servicios en segundo plano?
- a) **docker compose start**
 - b) **docker compose up -d --build**
 - c) **docker compose run --all**
 - d) **docker compose deploy**
6. ¿Qué función cumple la directiva **depends_on:** en la definición de un servicio dentro de Compose?
- a) Garantiza que la base de datos esté procesando consultas SQL antes de iniciar el contenedor web.
 - b) Establece la secuencia u orden de inicio de los contenedores (arrancar la BD antes que la app web).
 - c) Fusiona los archivos de código fuente de ambos contenedores.
 - d) Asigna la misma memoria RAM a ambos servicios.
7. ¿Dónde deben almacenarse las credenciales sensibles (usuarios, contraseñas de BD) en un proyecto con Docker Compose?
- a) Escritas directamente (hardcoded) en el archivo **docker-compose.yml**.
 - b) En un archivo local **.env** que debe estar excluido del control de versiones en el **.gitignore**.
 - c) En el archivo de documentación **README.md**.
 - d) En la carpeta pública del servidor web.
8. ¿Qué comando de Docker Compose detiene los contenedores y elimina las redes creadas por el proyecto?
- a) **docker compose stop**
 - b) **docker compose down**
 - c) **docker compose pause**

- d) `docker compose clean`

9. ¿Para qué sirve mantener un archivo `.env.example` en el repositorio público del proyecto?

- a) Para cifrar las variables de entorno en producción.
- b) Para servir como plantilla documentada de las variables que el proyecto requiere, sin revelar contraseñas reales.
- c) Para descargar las imágenes de Docker Hub.
- d) Para instalar Docker Compose en Linux.

10. ¿Qué comando de Compose permite monitorear los logs consolidando la salida de todos los servicios en tiempo real?

- a) `docker compose logs -f`
- b) `docker compose ps --logs`
- c) `docker compose inspect logs`
- d) `docker compose status -v`

11. ¿Qué diferencia existe entre ejecutar `docker compose down` y `docker compose down -v`?

- a) No hay ninguna diferencia.
- b) `docker compose down -v` además elimina los volúmenes nombrados del proyecto, borrando la información persistente de las bases de datos.
- c) `docker compose down -v` solo detiene la app web pero deja la base de datos activa.
- d) `docker compose down -v` activa el modo de diagnóstico detallado.

12. ¿Por qué la aplicación Flask dentro de su contenedor NO puede conectarse a PostgreSQL usando `host="localhost"`?

- a) Porque PostgreSQL rechaza todas las conexiones locales.
- b) Porque dentro del contenedor de Flask, `localhost` apunta a su propia interfaz de red loopback interna y no al contenedor de PostgreSQL.
- c) Porque Docker desactiva el protocolo TCP/IP.
- d) Porque `localhost` es una palabra reservada exclusiva de Windows.

Solucionario Explicado — Test Sesión 3

- 1-b: Docker Compose es la herramienta declarativa YAML para administrar aplicaciones multi-contenedor.
- 2-c: YAML exige indentación estricta con espacios. Usar tabuladores corrompe la lectura del archivo.
- 3-b: La sección `services:` define los contenedores que forman el stack del proyecto.
- 4-b: El DNS interno de Docker resuelve el nombre del servicio (`db`) a la IP interna del contenedor BD.
- 5-b: `docker compose up -d --build` compila si hay cambios y levanta todo en detached mode.
- 6-b: `depends_on` controla el orden de inicio de los procesos de contenedor.
- 7-b: Las credenciales deben guardarse en un archivo local `.env` no rastreado por Git.
- 8-b: `docker compose down` detiene contenedores y destruye las redes virtuales asociadas.
- 9-b: `.env.example` es la plantilla pública que guía a otros desarrolladores sobre las variables requeridas.
- 10-a: `docker compose logs -f` muestra la bitácora combinada en tiempo real (follow).
- 11-b: El flag `-v` ordena borrar los volúmenes nombrados, destruyendo los datos persistentes.
- 12-b: `localhost` dentro de un contenedor es su propia interfaz local, no la de otros contenedores del stack.

⚡ Quiz en Vivo de la Sesión 3 (6 Preguntas Rápidas para la Clase)

1. **(Quiz 3.1)** Si tu archivo `docker-compose.yml` tiene un error de espacio en la línea 5, ¿qué mensaje mostrará la terminal al ejecutar `docker compose up`?
 - a) `Connection refused`
 - b) `yaml: line 5: did not find expected key` o error de parsing YAML.
 - c) `Image not found`
 - d) `Password incorrect`
2. **(Quiz 3.2)** ¿Cuál es la diferencia entre la directiva `ports:` y la directiva `expose:` en Docker Compose?
 - a) `ports:` publica el puerto hacia la máquina Host (accesible desde el navegador); `expose:` solo deja el puerto visible para otros contenedores en la red interna.
 - b) `ports:` es para PostgreSQL y `expose:` es para Nginx.
 - c) Tienen la misma función exacta.
 - d) `expose:` se usa para conexiones por cable de fibra óptica.
3. **(Quiz 3.3)** Si deseas ingresar a la consola de la base de datos PostgreSQL en vivo dentro de Compose, ¿qué comando ejecutas?
 - a) `docker compose run db psql`
 - b) `docker compose exec db psql -U appuser -d appdb`
 - c) `docker connect db`
 - d) `docker compose logs db`
4. **(Quiz 3.4)** Si cambias una línea de código en tu `app.py` y corres `docker compose up -d` SIN la bandera `--build`, ¿qué ocurrirá?
 - a) Docker detectará el cambio y recompilará automáticamente.
 - b) Docker reutilizará la imagen previamente construida y NO verás reflejado tu cambio de código.
 - c) El comando fallará arrojando un error.
 - d) Se borrará la base de datos.
5. **(Quiz 3.5)** ¿Qué función cumple la variable de entorno `POSTGRES_PASSWORD` en el servicio de PostgreSQL?
 - a) Es la contraseña que usa Docker Desktop para instalarse.
 - b) Establece la contraseña del superusuario/usuario de la base de datos al inicializar el contenedor por primera vez.
 - c) Cifra el archivo `docker-compose.yml`.
 - d) Cambia la clave de tu cuenta de GitHub.
6. **(Quiz 3.6)** Si un compañero descarga tu proyecto de GitHub y ejecuta `docker compose up -d`, ¿por qué le dará error de conexión a la base de datos si olvidó crear el archivo `.env`?
 - a) Porque GitHub borra el código fuente.
 - b) Porque las variables `${POSTGRES_PASSWORD}` del YAML quedarán vacías y el motor de Postgres rechazará la inicialización.

- c) Porque Docker requiere internet pagado.
- d) Porque el archivo `Dockerfile` no se puede clonar.

Solucionario Explicado — Quiz Sesión 3

3.1-b: Los errores de indentación en YAML generan fallos de parsing al leer el documento.

3.2-a: `ports`: mapea al Host externo; `expose`: documenta el puerto para la red privada interna.

3.3-b: `docker compose exec <servicio> <comando>` ejecuta una utilidad interactiva en un contenedor activo.

3.4-b: Sin `--build`, Compose no reconstruye las imágenes locales aunque hayas editado archivos.

3.5-b: `POSTGRES_PASSWORD` es la variable obligatoria de arranque del contenedor PostgreSQL oficial.

3.6-b: Sin las variables leídas del `.env`, las credenciales sustituidas en el YAML quedan nulas.

SESIÓN 4: Redes, Volúmenes y Persistencia

Test de la Sesión 4 (12 Preguntas para Aula Virtual)

1. **¿Qué sucede con los datos escritos dentro del sistema de archivos de un contenedor si este se destruye y no tiene volúmenes configurados?**

- a) Se guardan automáticamente en la carpeta Documentos del usuario.
- b) Se pierden de forma permanente debido a la naturaleza efímera y volátil del almacenamiento del contenedor.
- c) Docker los almacena en un servidor de soporte en la nube.
- d) Se respaldan en un archivo `.zip` oculto.

2. **¿Cuál es la diferencia arquitectónica entre un Volumen Nombrado (Named Volume) y un Bind Mount?**

- a) Los volúmenes nombrados son gestionados por Docker en una ruta interna reservada del Host; los bind mounts enlazan una ruta o carpeta específica definida por el usuario.
- b) Los bind mounts borran los datos al apagar el equipo.
- c) Los volúmenes nombrados solo funcionan en sistemas Windows.
- d) Los bind mounts consumen el doble de memoria RAM.

3. **¿En qué ruta predeterminada del sistema de archivos de Linux gestiona Docker los Volúmenes Nombrados?**

- a) `/var/lib/docker/volumes/`
- b) `/etc/docker/storage/`
- c) `/tmp/docker/volumes/`
- d) `/home/user/.docker/data/`

4. **¿Qué comando permite inspeccionar la lista de contenedores conectados a una red virtual de Docker y conocer sus direcciones IP internas?**

- a) `docker network ls`
- b) `docker network inspect <nombre_red>`

- c) `docker network status`
- d) `docker ip show`

5. **¿Por qué es una recomendación clave de seguridad (Hardening) NO publicar el puerto de PostgreSQL (5432) en el archivo Compose de producción?**

- a) Porque PostgreSQL funciona más rápido sin puertos.
- b) Porque evita exponer el motor de base de datos a ataques o escaneos externos desde internet, permitiendo tráfico únicamente desde la red privada de Docker.
- c) Porque libera espacio en el disco duro.
- d) Porque inhabilita la necesidad de contraseñas.

6. **¿Cómo se realiza un respaldo (dump SQL) de una base de datos PostgreSQL en Compose sin necesidad de exponer su puerto al exterior?**

- a) Deteniendo el contenedor y copiando la carpeta `/var/lib/docker`.
- b) Ejecutando: `docker compose exec -T db pg_dump -U appuser appdb > backups/appdb.sql`.
- c) Abriendo una sesión en el navegador web.
- d) Descargando los archivos con `docker cp` mientras el contenedor está apagado.

7. **¿Por qué es necesario utilizar el flag `-T` al ejecutar `docker compose exec -T db pg_dump ...` dentro de un script de automatización?**

- a) Para ejecutar el respaldo en modo de prueba sin escribir datos.
- b) Para desactivar la asignación de un pseudo-TTY, evitando que caracteres de control de consola corrompan la redirección del archivo SQL.
- c) Para limitar el uso de CPU durante el respaldo.
- d) Para cifrar la salida con clave pública.

8. **¿Qué comando se debe utilizar para restaurar un respaldo SQL contenido en `backups/appdb.sql` hacia la base de datos PostgreSQL activa en Compose?**

- a) `docker compose exec -T db psql -U appuser appdb < backups/appdb.sql`
- b) `docker restore db < backups/appdb.sql`
- c) `docker import backups/appdb.sql db`
- d) `docker compose load -i backups/appdb.sql`

9. **¿Qué diferencia existe entre un contenedor en estado `running` y un contenedor en estado `healthy`?**

- a) `running` significa que el proceso principal arrancó; `healthy` indica que el Healthcheck confirmó que el servicio interno (ej. Postgres) ya está listo para responder tráfico.
- b) `healthy` es solo para contenedores Nginx.
- c) `running` significa que el contenedor consume más del 80% de CPU.
- d) No hay diferencia, son sinónimos.

10. **En la configuración de un Healthcheck para PostgreSQL, ¿qué herramienta de consola se utiliza habitualmente en la instrucción `test`?**

- a) `curl -f http://localhost`

- b) `pg_isready -U usuario -d dbnombre`
- c) `ping 127.0.0.1`
- d) `systemctl status postgresql`

11. Si un desarrollo requiere que los cambios de código fuente en la laptop se reflejen inmediatamente dentro del contenedor sin hacer `docker build`, ¿qué tipo de montaje se debe usar?

- a) Volumen Nombrado
- b) Bind Mount (ej. `./app:/app`)
- c) tmpfs mount
- d) Ninguno, se debe recompilar siempre.

12. ¿Qué comando de Docker elimina todos los volúmenes nombrados que NO estén asociados a ningún contenedor activo?

- a) `docker volume rm --all`
- b) `docker volume prune`
- c) `docker system clean`
- d) `docker volume reset`

Solucionario Explicado — Test Sesión 4

1-b: El almacenamiento interno del contenedor es capa efímera; al eliminarse el contenedor se pierden los datos.

2-a: Volúmenes nombrados = gestionados por Docker en su directorio interno; Bind Mounts = enlace directo a carpetas del host.

3-a: `/var/lib/docker/volumes/` es la ruta estándar en Linux donde Docker almacena los volúmenes nombrados.

4-b: `docker network inspect` detalla los contenedores adjuntos a una red y sus IPs asignadas.

5-b: Retirar `ports`: de la BD evita exponerla al exterior, manteniéndola accesible solo en la red interna.

6-b: `pg_dump` ejecutado vía `docker compose exec` extrae el backup en caliente directamente sobre la red privada.

7-b: El flag `-T` desactiva TTY, garantizando una redirección de flujo plana a archivo sin códigos de escape de terminal.

8-a: Redirigir el archivo SQL con `<` hacia `psql` ejecuta la restauración de las sentencias de la base de datos.

9-a: `running` confirma ejecución del proceso; `healthy` valida disponibilidad real mediante la prueba del Healthcheck.

10-b: `pg_isready` es la utilidad oficial de PostgreSQL para comprobar si el servidor acepta conexiones SQL.

11-b: Bind Mounts enlazan el sistema de archivos del host con el contenedor, permitiendo desarrollo en caliente (hot-reload).

12-b: `docker volume prune` limpia los volúmenes nombrados huérfanos sin contenedores vinculados.

Quiz en Vivo de la Sesión 4 (6 Preguntas Rápidas para la Clase)

1. (Quiz 4.1) Un alumno ejecutó `docker compose down -v`. ¿Qué ocurrió con la información de su base de datos PostgreSQL?

- a) La información fue enviada a la nube.
 - b) La información del volumen nombrado fue eliminada permanentemente por incluir la bandera `-v`.
 - c) La información se guardó en un archivo de respaldo.
 - d) No pasó nada, el comando fue ignorado.
2. **(Quiz 4.2)** Si tu aplicación web corre en el contenedor `web` y quieres verificar si puede resolver el nombre `db`, ¿qué comando de diagnóstico ejecutas?
- a) `docker compose exec web ping db`
 - b) `docker network status`
 - c) `docker logs db`
 - d) `docker inspect db`
3. **(Quiz 4.3)** ¿Por qué se prefiere un Volumen Nombrado en lugar de un Bind Mount para guardar los datos de PostgreSQL en producción?
- a) Porque los volúmenes nombrados ofrecen mejor rendimiento I/O y evitan problemas de permisos de usuario entre SO Host y Linux.
 - b) Porque los bind mounts son ilegales.
 - c) Porque los volúmenes nombrados son de solo lectura.
 - d) Porque ocupan menos RAM.
4. **(Quiz 4.4)** En la directiva de Healthcheck `retries: 5` e `interval: 10s`, ¿cuánto tiempo esperará Docker como máximo fallando la prueba antes de marcar el contenedor como `unhealthy`?
- a) 5 segundos.
 - b) 50 segundos aproximadamente (5 reintentos distanciados cada 10 segundos).
 - c) 10 minutos.
 - d) 2 segundos.
5. **(Quiz 4.5)** Si tu archivo de backup `appdb.sql` pesa 0 bytes después de ejecutar el comando de respaldo, ¿cuál fue la causa más probable?
- a) Que la base de datos tenía demasiada información.
 - b) Que el comando `pg_dump` falló por usuario/clave incorrecta o porque olvidaste el flag `-T` al redirigir la salida.
 - c) Que el disco duro de la laptop está lleno.
 - d) Que PostgreSQL se desinstaló.
6. **(Quiz 4.6)** Si deseas montar una carpeta local de forma que el contenedor solo pueda LEER sus archivos pero no modificarlos ni borrarlos, ¿cómo lo declaras en Compose?
- a) - `./archivos:/app:ro` (usando el flag Read-Only `:ro`)
 - b) - `./archivos:/app:lock`
 - c) - `./archivos:/app:secure`
 - d) - `./archivos:/app:write`

- 4.1-b: El flag `-v` en `docker compose down` destruye intencionalmente los volúmenes nombrados.
- 4.2-a: `exec web ping db` prueba en vivo la resolución DNS interna entre los servicios.
- 4.3-a: Los volúmenes nombrados son aislados y gestionados nativamente por el Docker Engine con alto rendimiento.
- 4.4-b: 5 reintentos con intervalo de 10s totalizan ~50s antes de cambiar la etiqueta a `unhealthy`.
- 4.5-b: Un dump de 0 bytes indica error de autenticación en la sentencia o falla de redirección por TTY.
- 4.6-a: El sufijo `:ro` configura el montaje en modo de solo lectura (Read-Only) para el contenedor.

SESIÓN 5: Docker en Producción

Test de la Sesión 5 (12 Preguntas para Aula Virtual)

1. **¿Por qué se coloca un servidor web Nginx como Reverse Proxy al frente de una aplicación web Flask en producción?**
 - a) Porque Flask no soporta código HTML.
 - b) Porque Nginx centraliza la seguridad, maneja TLS/SSL, sirve archivos estáticos y protege al servidor WSGI de Flask expuesto internamente.
 - c) Porque Nginx compila la base de datos PostgreSQL.
 - d) Porque reduce el consumo de memoria RAM a cero.
2. **En la configuración de Nginx (`default.conf`), ¿qué función cumple la directiva `proxy_pass http://web:5000;`?**
 - a) Descarga el código fuente de Flask desde GitHub.
 - b) Redirige de forma transparente las solicitudes HTTP entrantes hacia el puerto 5000 del servicio nombrado `web` en la red interna de Docker.
 - c) Bloquea el acceso a todas las direcciones IP externas.
 - d) Abre el puerto 5000 en la computadora del usuario final.
3. **¿Cuál es el beneficio de seguridad de configurar el servicio `web` con la directiva `expose: - "5000"` en lugar de `ports: - "5000:5000"` en Compose?**
 - a) Aumenta la velocidad de procesamiento de la CPU.
 - b) Garantiza que Flask solo sea accesible dentro de la red privada por Nginx, impidiendo que usuarios de internet accedan directamente al puerto 5000.
 - c) Permite ejecutar la app sin instalar Python.
 - d) Cifra el archivo `app.py`.
4. **En un `Dockerfile` Multi-stage, ¿qué realiza la línea `COPY --from=builder /root/.local /root/.local`?**
 - a) Copia archivos desde internet a la computadora del desarrollador.
 - b) Copia únicamente los paquetes y librerías instaladas en la etapa previa llamada `builder` hacia la etapa limpia de `runtime`.
 - c) Borra el historial de comandos de Linux.
 - d) Cifra la carpeta de usuario de la imagen.

5. **¿Qué sucede si un contenedor sin límites de recursos sufre una fuga de memoria (memory leak) en un servidor de producción?**

- a) Docker escala automáticamente el contenedor a otro servidor.
- b) Consumirá la memoria RAM del Host hasta que el Kernel de Linux active el **OOM Killer**, liquidando el proceso del contenedor con un Exit Code 137.
- c) El procesador aumentará su velocidad al doble.
- d) Se formateará el disco duro del servidor.

6. **¿Cómo se limitan los recursos de memoria RAM y procesador para un servicio en un archivo Compose?**

- a) En la sección `deploy.resources.limits` (ej. `memory: 256M` y `cpus: '0.5'`).
- b) Escribiendo comandos `chmod` en el Dockerfile.
- c) Configurando el archivo `.env`.
- d) Usando un Bind Mount.

7. **¿Qué utilidad ofrece configurar la rotación de logs en Compose con el driver `json-file` y las opciones `max-size: "10m"` y `max-file: "3"`?**

- a) Aumenta la velocidad del servidor Nginx.
- b) Previene que los archivos de bitácora crezcan indefinidamente y llenen el espacio total del disco duro del servidor.
- c) Borra los logs cada 10 minutos.
- d) Envía los logs por correo electrónico.

8. **¿Qué es `ttl.sh` en el ecosistema de contenedores?**

- a) Un comando de Linux para cambiar la hora del sistema.
- b) Un registro de imágenes Docker público y efímero que no requiere login y elimina automáticamente las imágenes tras un tiempo definido (ej. 24h).
- c) Un plugin para compilar código Python.
- d) Un firewall de red para Compose.

9. **¿Cuál es el nombre del registro oficial de contenedores integrado en la plataforma de GitHub?**

- a) `docker.io`
- b) `ghcr.io` (GitHub Container Registry)
- c) `registry.github.com`
- d) `hub.github.io`

10. **¿Cuál es la forma segura de autenticarse en la CLI de Docker contra `ghcr.io` o Docker Hub en pipelines automatizados?**

- a) Escribiendo la contraseña personal en texto plano en la consola.
- b) Usando un Personal Access Token (PAT) enviado mediante la entrada estándar con `--password-stdin`.
- c) Desactivando la autenticación de usuario.
- d) Subiendo la clave privada SSH al repositorio público.

11. ¿Qué comando de la consola de Docker permite monitorear el consumo de CPU, RAM y red de todos los contenedores activos en tiempo real?
- a) `docker stats`
 - b) `docker top`
 - c) `docker inspect`
 - d) `docker system df`
12. ¿Por qué es importante agregar la instrucción `USER appuser` al final de un `Dockerfile` de producción?
- a) Para acelerar el tiempo de booteo de la app.
 - b) Para que la aplicación se ejecute con un usuario sin privilegios, evitando que un atacante obtenga acceso root al sistema host en caso de una vulnerabilidad.
 - c) Para permitir el acceso SSH al contenedor.
 - d) Para habilitar la instalación de paquetes en caliente.

🔑 Solucionario Explicado — Test Sesión 5

- 1-b: Nginx actúa como Reverse Proxy administrando SSL, estáticos, seguridad y balanceo hacia el backend.
- 2-b: `proxy_pass` redirige las peticiones entrantes al puerto y hostname interno del servicio web.
- 3-b: `expose` restringe el puerto a la red interna de Docker, evitando ataques directos desde el host/internet.
- 4-b: `COPY --from=builder` extrae solo los artefactos necesarios compilados en una etapa previa descartable.
- 5-b: Sin límites, la fuga de RAM activa el Out-Of-Memory (OOM) Killer del Kernel finalizando el proceso (Exit 137).
- 6-a: `deploy.resources.limits` establece las cotas máximas de CPU y RAM que un contenedor puede consumir.
- 7-b: Limitar el tamaño de logs (`max-size` y `max-file`) evita desbordamientos de almacenamiento en disco.
- 8-b: `ttl.sh` es un registro efímero anónimo excelente para pruebas temporales de integración continua.
- 9-b: `ghcr.io` es el registro de contenedores de GitHub altamente integrado con GitHub Actions.
- 10-b: Pasar un Personal Access Token (PAT) vía `--password-stdin` previene fugas de credenciales en los logs.
- 11-a: `docker stats` entrega métricas en vivo de consumo de CPU, RAM, I/O y ancho de banda de red.
- 12-b: Principio de menor privilegio: correr como usuario no-root (`USER appuser`) minimiza el impacto de exploits.

⚡ Quiz en Vivo de la Sesión 5 (6 Preguntas Rápidas para la Clase)

1. (Quiz 5.1) Si tu navegador muestra el error `502 Bad Gateway` al intentar abrir `http://localhost:8080`, ¿cuál es la causa más probable?
- a) Que PostgreSQL no tiene contraseña.
 - b) Que Nginx está activo en el puerto 8080, pero la aplicación Flask en el servicio `web` aún no arranca o la directiva `proxy_pass` tiene un puerto/nombre incorrecto.
 - c) Que se borró el disco duro de la laptop.
 - d) Que el archivo `.dockerignore` está dañado.
2. (Quiz 5.2) ¿Cuál es el orden estricto de Troubleshooting antes de modificar cualquier archivo de código?

- a) Modificar el Dockerfile -> Reiniciar la PC -> Leer los logs.
 - b) Verificar Estado (`compose ps`) -> Leer Logs (`compose logs`) -> Inspeccionar Recursos (`stats`) -> Inspeccionar Red/Variables (`inspect`).
 - c) Borrar todos los volúmenes -> Reinstalar Docker.
 - d) Subir el código a GitHub -> Esperar.
3. **(Quiz 5.3)** Si compilas un Dockerfile con Multi-stage y el tamaño de la imagen final pasa de 900MB a 120MB, ¿qué beneficio obtuviste además de ahorrar espacio?
- a) La pantalla del navegador cargará en color azul.
 - b) Se redujo la superficie de ataque y los tiempos de descarga/despliegue en la nube se aceleraron drásticamente.
 - c) La base de datos PostgreSQL responderá el doble de rápido.
 - d) Ya no se necesitará usar contraseñas.
4. **(Quiz 5.4)** ¿Qué flag de montaje en Compose garantiza que el archivo de configuración `default.conf` de Nginx no pueda ser alterado por el contenedor en ejecución?
- a) `:ro` (Read-Only)
 - b) `:rw` (Read-Write)
 - c) `:lock`
 - d) `:secure`
5. **(Quiz 5.5)** Si ejecutas `docker build --target builder -t mi-app:build .`, ¿qué etapa del Dockerfile Multi-stage se compilará?
- a) Todo el Dockerfile completo.
 - b) Únicamente la etapa nombrada `builder`, deteniéndose ahí sin generar la etapa final de runtime.
 - c) Ninguna, el comando falla.
 - d) Solo la etapa de Nginx.
6. **(Quiz 5.6)** ¿Por qué se utiliza Gunicorn (CMD `["gunicorn", "-w", "2", "-b", "0.0.0.0:5000", "app:app"]`) en producción en lugar del servidor por defecto de Flask (`python app.py`)?
- a) Porque Flask es de pago.
 - b) Porque el servidor interno de Flask es solo para desarrollo; Gunicorn es un servidor WSGI de producción capaz de manejar múltiples peticiones concurrentes con workers.
 - c) Porque Gunicorn convierte Python a JavaScript.
 - d) Porque Gunicorn no requiere memoria RAM.

Solucionario Explicado — Quiz Sesión 5

- 5.1-b: El error 502 indica que el Reverse Proxy Nginx no puede comunicarse con el upstream (`web:5000`).
- 5.2-b: Protocolo de diagnóstico: medir estado (`ps`), revisar errores (`logs`), recursos (`stats`) y config (`inspect`).
- 5.3-b: Menor peso implica menos vulnerabilidades (CVEs) y despliegues más veloces en clusters.
- 5.4-a: `:ro` monta el volumen en modo de solo lectura para evitar tampering accidental o malicioso.
- 5.5-b: `--target` detiene la compilación en una etapa intermedia específica del Dockerfile.
- 5.6-b: Werkzeug (servidor dev de Flask) es mono-hilo e inseguro; Gunicorn es un servidor WSGI robusto con workers.

SESIÓN 6: Proyecto Final y Despliegue Completo

Test de la Sesión 6 (12 Preguntas para Aula Virtual)

1. **¿Qué ventaja técnica ofrece la estrategia de separar los archivos Compose en `compose.yml` (base) y `compose.prod.yml` (override de producción)?**
 - a) Permite ejecutar diferentes versiones del motor de Docker en la misma computadora.
 - b) Reutiliza la arquitectura base común y aplica únicamente las reglas de producción (imágenes estáticas, restart policies, Nginx) sin duplicar código YAML.
 - c) Cifra el código fuente en formato binario.
 - d) Elimina la necesidad de usar imágenes base.
2. **Al compilar imágenes repetidamente durante el desarrollo, ¿qué residuo consume espacio en disco silenciosamente?**
 - a) Volúmenes de bases de datos duplicados.
 - b) Capas e imágenes antiguas huérfanas sin etiqueta conocidas como "dangling images" (`<none>:<none>`).
 - c) Archivos de registro `.log` en el sistema operativo.
 - d) Caché del navegador web.
3. **¿Qué comando de la CLI de Docker te permite auditar el consumo total de disco detallado por imágenes, contenedores, volúmenes y caché?**
 - a) `docker stats`
 - b) `docker system df`
 - c) `docker volume ls`
 - d) `docker inspect --size`
4. **Si deseas eliminar contenedores detenidos, redes sin uso e imágenes huérfanas sin tag de forma segura, ¿qué comando ejecutas?**
 - a) `docker system clean`
 - b) `docker system prune`
 - c) `docker container rm --all`
 - d) `docker image rm --force`
5. **¿Qué riesgo crítico se debe considerar antes de ejecutar `docker system prune -a --volumes` en un servidor de producción?**
 - a) Que borrará los archivos de código del sistema host.
 - b) Que eliminará TODOS los volúmenes nombrados no asociados a contenedores activos en ese instante, pudiendo causar pérdida total de datos de BDs inactivas.
 - c) Que detendrá el servicio de internet.
 - d) Que desinstalará la consola de Docker.

6. ¿Por qué se utilizan scripts automatizados en Bash como `desplegar.sh` para ejecutar las actualizaciones en servidores?

- a) Porque aumentan la velocidad de renderizado de la aplicación web.
- b) Porque estandarizan el flujo sin errores manuales: actualización de código, apagado, reconstrucción y limpieza segura en una sola secuencia.
- c) Porque protegen el servidor contra virus.
- d) Porque evitan la necesidad de usar contraseñas en Git.

7. Al crear un script Bash de despliegue en Windows para ejecutar en un servidor Linux, ¿por qué motivo suele fallar con el error : `command not found`?

- a) Por incompatibilidad de versiones de Python.
- b) Porque los editores de Windows guardan los saltos de línea con formato `CRLF` (`

), mientras que Linux requiere formato `LF` (`

- c) Porque Linux no soporta scripts Bash.
- d) Porque las variables de entorno están cifradas.

8. ¿Qué utilidad de consola se debe utilizar para convertir los saltos de línea de un script de formato Windows (`CRLF`) a formato Linux (`LF`)?

- a) `dos2unix`
- b) `unix2dos`
- c) `chmod +x`
- d) `tar -xzf`

9. ¿Qué realiza la instrucción de seguridad `set -euo pipefail` al inicio de un script Bash de despliegue?

- a) Activa el modo interactivo con el usuario.
- b) Detiene la ejecución del script inmediatamente si un comando falla (`-e`), si se usa una variable no definida (`-u`) o si falla una tubería (`-o pipefail`).
- c) Ejecuta los comandos en paralelo en múltiples procesadores.
- d) Oculta la salida de los comandos en la pantalla.

10. Si deseas obtener la dirección IP interna asignada a un contenedor desde un script Bash, ¿qué comando ejecutas?

- a) `docker logs app-web`
- b) `docker inspect --format '{{.NetworkSettings.IPAddress}}' app-web`
- c) `docker stats app-web --no-stream`
- d) `docker network show app-web`

11. ¿Cómo puedes auditar las bitácoras del servicio `web` generadas exclusivamente en los últimos 15 minutos?

- a) `docker compose logs --tail=15 web`
- b) `docker compose logs --since=15m web`
- c) `docker stats web --time=15`

- d) `docker inspect web --logs=15`

12. ¿Cuál es la prueba de fuego que debe cumplir el Proyecto Final de arquitectura en Docker?

- a) Que la aplicación ocupe menos de 1 MB.
- b) Que la infraestructura completa (Flask + PostgreSQL + Nginx + Volúmenes) pueda ser destruida y reconstruida desde cero de forma 100% reproducible mediante un solo script.
- c) Que corra únicamente en computadoras de la marca Apple.
- d) Que no use archivos `.env`.

🔑 Solucionario Explicado — Test Sesión 6

1-b: Combinar manifiestos con `-f` permite aplicar overrides de producción reutilizando la base común.

2-b: Las compilaciones sucesivas generan capas sueltas huérfanas descartables (`dangling images`).

3-b: `docker system df` presenta el desglose detallado de uso de espacio por tipo de objeto de Docker.

4-b: `docker system prune` limpia contenedores detenidos, redes en desuso y capas huérfanas.

5-b: Agregar `--volumes` a `system prune` es peligroso porque borra volúmenes no asociados a contenedores vivos.

6-b: Los scripts de despliegue estandarizan el flujo de entrega continua garantizando ejecuciones sin error humano.

7-b: Los caracteres `CRLF` de Windows colocan retornos de carro invisibles que corrompen el intérprete de Linux.

8-a: `dos2unix` convierte la codificación de fin de línea de archivos de texto al estándar POSIX/Linux (`LF`).

9-b: `set -euo pipefail` es la bandera de modo estricto en Bash que aborta la ejecución ante cualquier fallo.

10-b: `docker inspect` con plantillas Go (`--format`) extrae atributos JSON específicos como la dirección IP.

11-b: `--since=15m` filtra los registros de logs producidos en los últimos 15 minutos.

12-b: La meta fundamental del curso es la reproducibilidad completa e inmutable de la infraestructura.

⚡ Quiz en Vivo de la Sesión 6 (6 Preguntas Rápidas para la Clase)

1. **(Quiz 6.1)** ¿Cómo ejecutas Docker Compose aplicando al mismo tiempo el archivo base `compose.yml` y el archivo de producción `compose.prod.yml`?

- a) `docker compose -f compose.yml -f compose.prod.yml up -d`
- b) `docker compose run compose.yml compose.prod.yml`
- c) `docker build compose.yml`
- d) `docker compose merge`

2. **(Quiz 6.2)** Si ejecutas `chmod +x desplegar.sh`, ¿qué permiso le acabas de otorgar al archivo script en Linux?

- a) Permiso de lectura pública.
- b) Permiso de ejecución (`+x`) para poder correr el script con `./desplegar.sh`.
- c) Permiso de cifrado de contraseñas.
- d) Permiso de eliminación automática.

3. **(Quiz 6.3)** ¿Qué hace el flag `--remove-orphans` al ejecutar `docker compose down --remove-orphans`?

- a) Borra la base de datos PostgreSQL.
- b) Elimina los contenedores de servicios que pertenecían al proyecto pero que fueron removidos del archivo YAML actual.
- c) Elimina los archivos del código fuente.
- d) Cierra la sesión de usuario de Docker Hub.

4. **(Quiz 6.4)** Si deseas ver las últimas 50 líneas del log del servicio web sin quedarte enganchado a la consola, ¿qué comando utilizas?

- a) `docker compose logs --tail=50 web`
- b) `docker compose logs -f web`
- c) `docker status web 50`
- d) `docker inspect web --lines 50`

5. **(Quiz 6.5)** Si tu script de producción ejecuta `cp .env.prod .env`, ¿por qué se realiza este paso?

- a) Para que Compose lea las variables de entorno de producción desde el archivo `.env` que utiliza por defecto.
- b) Para borrar el archivo de Python.
- c) Para subir las claves a GitHub.
- d) Para cambiar la clave de la laptop.

6. **(Quiz 6.6)** ¿Cuál es el puerto de entrada final por el cual los clientes accederán a la aplicación en la arquitectura de producción de nuestro curso?

- a) Puerto 5000 (Flask)
- b) Puerto 5432 (PostgreSQL)
- c) Puerto 8080 / 80 (Nginx Reverse Proxy)
- d) Puerto 22 (SSH)

Solucionario Explicado — Quiz Sesión 6

6.1-a: Múltiples banderas `-f` combinan secuencialmente los archivos manifest de Compose.

6.2-b: `chmod +x` otorga permisos de ejecución al script Bash en sistemas POSIX/Linux.

6.3-b: `--remove-orphans` limpia servicios obsoletos que quedaron de versiones previas del YAML.

6.4-a: `--tail=50` imprime únicamente las últimas 50 líneas finalizando el comando inmediatamente.

6.5-a: Reemplazar `.env` con `.env.prod` asegura que Compose inyecte los parámetros del entorno de producción.

6.6-c: El Reverse Proxy Nginx en el puerto 8080 (o 80) es el único punto de entrada público expuesto hacia el Host.